

AI 辅助内核开发 多智能体工作流汇报

探索 → 规划 → 生成

审核 ↔ 调试

Patch Ready

面向 Linux 内核 / RISC-V / KVM 等高上下文、高验证成本场景
将“问题发现到补丁准备”组织为可审计、可迭代、带人工闸门的工程系统

探索

规划

生成

审核

调试

汇报内容：方案概览、架构、迭代闭环、示例、落地建议

为什么需要这套 workflow

核心判断：内核开发不是“让一个模型写代码”，而是把高风险认知活动拆成可验证的职责链。

当前挑战

内核语义强依赖架构背景、邮件列表上下文和历史设计取舍

一次性大任务容易导致 token 开销、实现偏航和审查风险同步放大

“能编过”不等于“能被上游接受”

设计原则

小问题切分，单轮只处理一个窄议题

状态与证据外置为工件，避免上下文膨胀

关键节点保留人工 Gate

目标结果

建立 discover → plan → generate → review → debug → patch 的闭环

形成可审计、可重试、可复盘的研发流水线

把人工时间集中在高价值判断上

5

核心智能体层级

3

人工闸门

2+

生成 / 审核迭代轮次

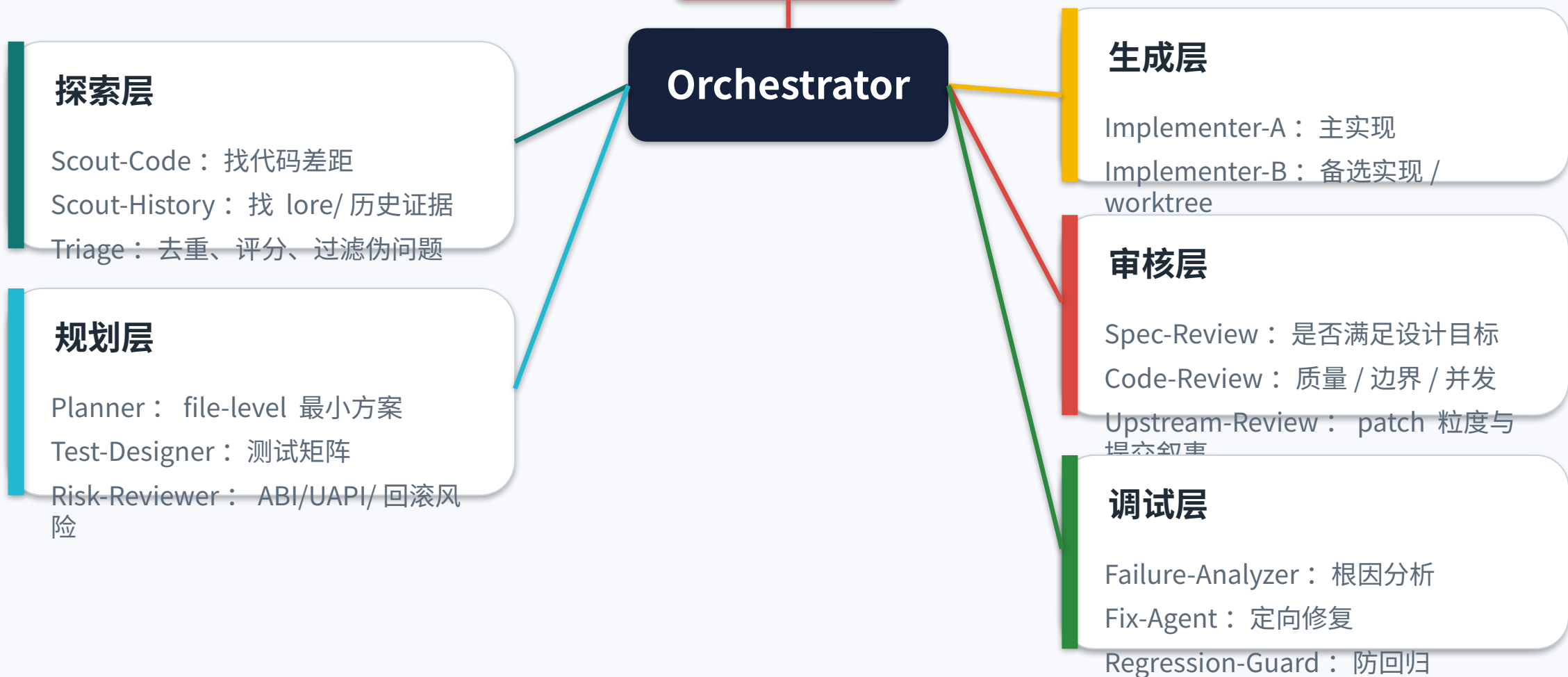
1

每轮仅 1 个 issue / 分支 / 目标

结论：AI 更适合承担“探索、方案生成、验证、归因、修复”的模块化工作，而不是跳过流程直接投递上游。

多智能体总体架构

控制中枢负责编排；探索、规划、生成、审核、调试五 **Human Gates** 只在高风险节点审批。



关键约束：同一 issue 在生成阶段只能有一个主写分支；审核可并行；调试必须串行回到审核。

五阶段主线：探索 → 规划 → 生成 → 审核 → 调试

探索

发现真实问题

规划

形成最小方案

生成

产出代码 / 日志

审核

独立质量判定

调试

失败闭环与回归防护

探索

输入：源码、架构目录、lore、历史提交

输出：代码证据、历史证据、gap_registry

DoD：问题真实、范围清晰、可转为窄issue

规划

输入：gap 条目与证据 bundle

输出：design / test matrix / risk

DoD：文件级路径清晰、可执行、可回滚

生成

输入：方案与测试矩阵

输出：代码修改、构建日志、测试日志、draft patch

DoD：最小实现可被 review 消费

审核

输入：实现结果与日志工件

输出：spec/code/upstream 三类评审

DoD：PASS / REVISE / DEBUG / BLOCKED

调试

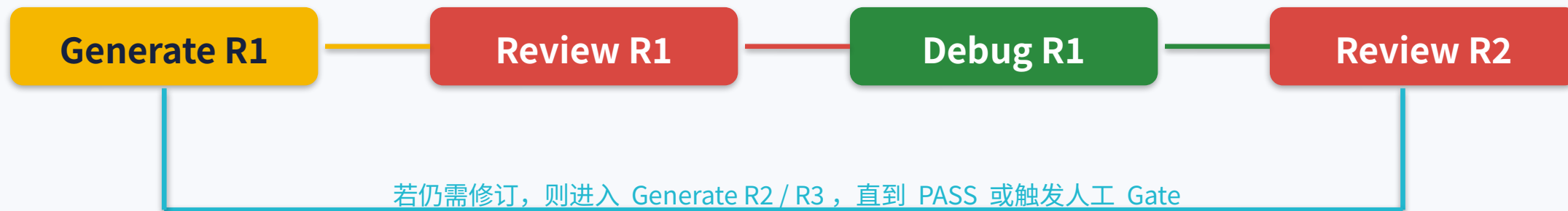
输入：失败日志与审查意见

输出：根因报告、修复摘要、回归结果

DoD：回到审核而不是直接放行

核心机制：生成与审核的多轮迭代

原则：生成不是一次成稿，审核也不是终点判决；两者之间必须允许“修订生成 ↔ 再审核”的多轮来回。



每轮必须记录什么

round / issue_id / 输入工件路径
本轮目标、修改文件、构建 / 测试结果
review decision 与 next action

停止条件

连续 3 轮无实质收敛
同一错误反复出现且根因不稳定
出现架构语义争议或回归持续增加

价值

降低“一次性过度设计”风险
把失败从黑盒变成结构化反馈
让人工只介入真正难判的地方

示例：demo-riscv-kvm-mmukconfig

演示问题：让 CONFIG_RISCV_KVM 的 MMU 依赖在 Kconfig dependency 与 help text 中都显式表达。



这个示例证明：调试不一定来自编译失败，也可以来自“规格审查指出需求未闭环”。

工件体系、质量闸门与调度建议

关键工件目录

artifacts/<issue>/discover : 代码 / 历史证据
plans : design / test-matrix / risk
run_history + logs : 每轮执行与验证记录
review : spec/code/upstream 审查结论
debug : failure-analysis / fix / regression
patches : draft patch / cover letter / checkpatch

三个人工 Gate

Gate-1 : 问题是否真实
Gate-2 : 方案是否最小可行
Gate-3 : patch/cover/ 收件人是否合规

自动质量闸门

make ARCH=riscv
defconfig / build
kselftest / kunit / QEMU 板卡验证
checkpatch.pl /
get_maintainer.pl

调度建议

discovery-job : 定时发现 gap
planning-job : issue_created 后生成方案
generation-job : design_ready 后产出代码
review-job : 独立并行审查
debug-job : review 或 test failed 后进入修复

建议: 每个 issue 独立目录、独立状态机、独立 worktree , 所有 Agent 只读取必要摘要与路径, 不吞整段历史上下文。

落地路径与汇报结论

MVP 落地建议

先只跑 1 个高置信度窄 issue
至少经历 2 轮生成 - 审核迭代
至少经历 1 次调试 - 再审核闭环
生成 patch-ready 材料，但保留人工最终发信

下一步工程化

补 workflow.yaml / 目录模板
固化 Claude Code / Codex 角色 prompts
脚本化 checkpatch / 日志归档 / 状态回写
加入预算控制与失败阈值

最终结论

多智能体价值不在“多开模型”，
而在职责解耦与相互制衡
生成与审核必须多轮迭代
调试必须回流审核，防止“修完即过”
人工应只在高价值判断点介入

**半自动 · 强审计 · 可迭代 · 面向
内核贡献**